Workshop Python Image Analysis

Martijn Wehrens, 2026-04

Estimated time: 20 mins presenting

Chapter IIB: How computers represent numbers

Data types determine what information can be stored

Types of data

Multiple data types exist. There are three important distinctions between these types:

- The bit-depth;
  - Determines how many values can be stored, e.g. 0-255 values for a bit depth of 8 (unsigned).
  - More depth takes more disk space.
  - **Relevance:** especially when generating large amounts of images, a trade-off between storage requirements and value range becomes important. Typically, you'll use 8-bit or 16-bit images.
- Signed or unsigned;
  - Whether negative values can be stored.
  - **Relevance:** usually only positive numbers are used in images, so you won't see the 'signed' format often.
- Integer vs. Floating Point;
  - Integer can only store whole numbers.
  - Floating Point is a technically more complicated format, which can store (very large and very small) numbers with a decimal point, such as 2.0005, 3.14159, 100023.93, et cetera.

– **Relevance:** Floating point images are often used in intermediate steps of image processing, but are rarely used for storing raw images.

The table below summarizes common image data formats. In practice, **uint8** and **uint16** are used most frequently.

Type	Bit-depth	dtype	Values
Unsigned integer	8	`uint8`	0 to 255
Signed integer	8	`int8`	-128 to 127
Unsigned integer	16	`uint16`	0 to 65535
Signed integer	16	`int16`	-32768 to 32767
Unsigned integer	32	`uint32`	0 to 4,294,967,295
Signed integer	32	`int32`	-2,147,483,648 to 2,147,483,647
Floating point	32	`float32`	-3.4E+38 to 3.4E+38 *
Floating point	64	`float64`	**

```
import matplotlib.pyplot as plt
import seaborn as sns
import tifffile as tiff
import numpy as np
```

Difference between 8-bit and 16-bit sometimes important

```
# Load a picture of fluorescently labeled HeLa cells (8 bit)
img_path8bit = 'images/biological/exampleHeLa-8bit.tif'
img_HeLa_8bit = tiff.imread(img_path8bit)

# Display information about the image
print('The type of this image is: ', img_HeLa_8bit.dtype.name)
print('uint8 has ', 2**8, ' possible values')

# Show the image
_ = plt.imshow(np.log(img_HeLa_8bit+1), cmap='viridis')
```

```
_ = plt.imshow(np.log(img_HeLa_8bit[0:250,1575:1700]+1),
               cmap='viridis')
```

```
# Load a picture of fluorescently labeled HeLa cells
img_path = 'images/biological/exampleHeLa.tif'
img_HeLa = tiff.imread(img_path)

# Display information about the image
print('The type of this image is: ', img_HeLa.dtype.name)
print('uint16 has ', 2**16, ' possible values')

_ = plt.imshow(np.log(img_HeLa[0:250,1575:1700]+1),
               cmap='viridis')
```

Take home messages

- When acquiring data:
  - Decide the relevant biological range and amount of detail within that range that you need.
  - Make sure the file format you save to allows for that range and level of detail.
    - Histograms can help during that assessment.
- Generally, opt for the uint16 format and save your images as .tif to prevent losses.
- Other pitfalls:
  - Saturated pixels:
    - Occurs when your real signal falls outside your file format range.
    - Values higher than the maximum value will collapse to the maximum.
    - Example: Say you measure a photon count of 466, but your storage range is 0-255, then the value 466 will be stored as 255. Information is lost.

Let's get real (size)

```
# Example image
img = tiff.imread('images/biological/microcolony_ecoli.tif')
_ = plt.imshow(img, cmap="gray")
_ = plt.xlabel('x [pixels]'); _ = plt.ylabel('y [pixels]')
```

```
# Luckily, we know the size of a pixel
pixel_size = 0.041  # µm per pixel
    # In many modern setups, this is stored in the metadata
    # E.g. FIJI > Image > Show Info

# Get the number of pixels per dimension
nx = img.shape[1]
ny = img.shape[0]

# Define extent: [xmin, xmax, ymin, ymax]
extent = [0, nx * pixel_size, 0, ny * pixel_size]

# Make the plot
plt.imshow(img, extent=extent, origin="lower", cmap="gray")
plt.xlabel("x [µm]")
plt.ylabel("y [µm]")
plt.show()

# This is a specific example, but determining lengths
# or areas can often be relevant, in which case you
# need to know the pixel size.
```

```
# Note the confusing x,y vs. img[i,j] conventions

# generate 100 x 500 image with noise
img_example = np.random.random((100, 500))
plt.imshow(img_example)

# now set extent explicitly
plt.imshow(img_example, extent=[0,5,0,1])
```

Type conversions

```
# what happens here?
plt.title('naive conversion to uint8')
plt.imshow(img.astype('uint8'))
plt.show()

plt.title('Original histogram')
plt.hist(img.ravel(), bins=256)
```

```
plt.show()

plt.title('After conversion to uint8')
plt.hist(img.astype('uint8').ravel(), bins=256)
plt.show()

# OPTIONAL
# .. because cyclical indexing
# .. this equals ..
plt.imshow(img % 256)

# So rescale first
from skimage import exposure
img_rescaled8bit = exposure.rescale_intensity(img, out_range=(0, 255)).astype('uint8')
    # could also use shortcut out_range='uint8'
    # img_rescaled8bit = exposure.rescale_intensity(img, out_range='uint8')
plt.title('After proper conversion to uint8')
plt.imshow(img_rescaled8bit)
plt.show()
plt.title('After proper conversion to uint8')
plt.hist(img_rescaled8bit.ravel(), bins=256)
plt.show()
```

Lossy and lossless data storage

```
# reading/writing jpg, tiff, png, etc
import imageio as iio
    # imageio
    #  - collection of plugins
    #  - supports many formats
    #  - no setting, selects lib through prioritized table:
    #       - https://imageio.readthedocs.io/en/stable/formats/index.html
    #       - tiffile is #1 for tiff
    # (skimage.io is deprecated; was a wrapper around iio anyways)

iio.imsave('output-test/exampleHeLa_05.jpg', img_rescaled8bit, quality=5)
iio.imsave('output-test/exampleHeLa_50.jpg', img_rescaled8bit, quality=50)
iio.imsave('output-test/exampleHeLa_100.jpg', img_rescaled8bit, quality=100)

# Inspect the output
```

```
# Can you explain what happened?

# jpg (lossy) will lead to imprecise storage of your image data
# tiff (lossless) will compress, but precisely store data values

# Would you save your data as jpg, at all?
```

For further reading, check out lossless and lossy compression at Wikipedia.

See imageio's documentation for a list of supported formats, and which libraries are preferentially used to read files.